

CS309 Lab 4 ARM Program Basic

Objective: The objective of this assignment is to get the students to learn the basics of ARM Assembly. This includes: output, input, comparisons, basic arithmetic, and loop controls. Students need to properly document their code and demonstrate they know how to use the GNU debugger.

Part I: ARM Assembly Code Requirements

Prompt the user to enter a number between 1 and 12. **Reject** any invalid user inputs and terminate the program.

From 1 to the user entered number print out the number and the product (*) of all the integers to that point. This is the factorial of the number.

Terminate the program when all the numbers from 1 to the user entered number are printed.

Example of what the output should look like:

This program will print the factorial of the integers from 1 to a number you enter. Please enter an integer number from 1 to 12.

You entered 4.

Following is the number and the product of the integers from 1 to n.

Number	n!
1	1
2	2
3	6
4	24

Your program needs to be well documented. Points will be deducted for code which is poorly documented. See the Student Lab Handbook for expected code comments.

Students need to have a file which, at the very least contain the following:

1. Proper header information that include commands that assemble, link, run the file and run the file using the debugger that is specific for their file.
2. Data section that defines all the output strings.
3. Code comment sections that describe the major sections of the code with the output loop clearly identified.

Part II: GNU Debugger

Refer to the Student Lab Handbook Appendix F on how to run the debugger or the Canvas module on the Raspberry Pi Zero. Students are to complete the grading rubric in this document and turn it in with their ARM assembly file (.s):

CS309 Lab 4 ARM Program Basic

This program was assigned:

Spring 2019

Spring 2021

Spring 2022

CS309 Lab 4 ARM Program Basic

Grading Rubric

Student _____ Lab Instructor: _____

Total Score: _____ Late Penalty? No _____ Yes: _____

Feature	Points off
Code Comments/Documentation - Class, Term, Author, Date. - Purpose of software - Documentation for the start of each loop. - Documentation for error checking - In-line comments that explain why the following code exists.	
Welcome and instruction messages are displayed and clear.	
Does the program work for an input of 1?	
Does the program work for an input of 12? The correct final product for 12 is 479,001,600.	
Does the program work for a number between 1 and 12?	
Does the program reject a number less than 1?	
Does the program reject a number greater than 12?	
Does the program reject a string or character input (i.e., hello, A)	
Debugger commands need to be demonstrated by the student	
Provide the starting address for the first string that is printed to the screen. The address in hex is: _____ Note: This can be checked by setting a breakpoint prior to the printf call and checking the contents of r0.	
Set a breakpoint at the top of the loop that calculates and prints the numbers.	
Examine the contents of the registers that are being used for the counter and the product. My counter register is _____ and the content is _____ My product register is _____ and the content is _____	

CS309 Lab 4 ARM Program Basic

Step through the loop one time and reexamine the contents of the registers being used for the counter and the product to show they have been updated correctly. My counter register is _____ and the content is _____ My product register is _____ and the content is _____	
Set a breakpoint at the exit point of the loop once all the calculations are complete. Reexamine the contents of the registers being used for the counter and the product to show they have been updated correctly and have the correct final values. My counter register is _____ and the content is _____ My product register is _____ and the content is _____	
Set a breakpoint where the last string is printed by your program. Provide the address that is loaded into r0 prior to the call to printf. The address in hex is: _____ Set a breakpoint where the code returns from the printf. Provide the value that is now in r0 after the call to printf. The value in hex in r0 is: _____ Note: The previous value for r0 should have changed after the call to printf.	